Basics Day 10 — Session 10 (Hours 37–40)

Python Programming (Basic) • Intro to Object-Oriented Programming

Session 10 Overview

- Hour 37: Intro to classes — objects, attributes, methods
- Hour 38: Collections of objects + searching
- Hour 39: Basic encapsulation + data validation
- Hour 40: Checkpoint 5 — Functions, modules & intro OOP

Hour 37: Intro to Classes — Objects, Attributes, Methods

Learning Outcomes

- Create a basic class with `__init__`
- Define instance attributes using `self`
- Write and call methods on objects
- Instantiate multiple objects from one class

Why Object-Oriented Programming?

The Problem With Plain Dictionaries

```
# Contact as a plain dictionary — hard to enforce structure
contact1 = {"name": "Alice", "phone": "555-1234"}
contact2 = {"name": "Bob",   "phone": "555-5678"}
contact3 = {"name": "Carol"}   # oops — forgot phone entirely

# Every caller must know the exact key names
print(contact1["name"])    # works
print(contact3["phone"])   # KeyError — no phone key!
```

The Object-Oriented Solution

```
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name
        self.phone = phone
```

The Blueprint Analogy

Class = Blueprint, Object = Built Thing

Concept	Real World Analogy	Python
Class	Architectural blueprint	`class Contact:`
Object / Instance	An actual built house	`c = Contact("Alice", "555-1234")`
Attribute	Room dimensions, color	`c.name`, `c.phone`
Method	What you can do in/with the house	`c.display()`

Key Insight

- The **class** is defined once.

Anatomy of a Class

The class Statement

```
class Contact:                                  # 1. class keyword + name
    def __init__(self, name: str,               # 2. constructor — auto-called
                 phone: str) -> None:
        self.name = name                        # 3. instance attribute
        self.phone = phone                      # 3. instance attribute

    def display(self) -> str:                  # 4. method — first param is self
        return f"{self.name}: {self.phone}"
```

Breaking It Down

Part	Purpose
`class Contact:`	Defines the blueprint — use `PascalCase` for class names

Understanding `self`

What Is `self`?

```
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name    # "this object's name = the name passed in"
        self.phone = phone  # "this object's phone = the phone passed in"

    def display(self) -> str:
        # self.name refers to THIS object's name attribute
        return f"{self.name}: {self.phone}"
```

How Python Uses `self` Behind the Scenes

```
c1 = Contact("Alice", "555-1234")
c2 = Contact("Bob",   "555-5678")

# When you write:
print(c1.display())
```

Creating and Using Objects

Instantiation — Building an Object From the Blueprint

```
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name
        self.phone = phone

    def display(self) -> str:
        return f"{self.name}: {self.phone}"

# Create objects — each is independent
c1 = Contact("Alice", "555-1234")
c2 = Contact("Bob",   "555-5678")
c3 = Contact("Carol", "555-9999")

# Access attributes directly
print(c1.name)    # Alice
print(c2.phone)   # 555-5678

# Call methods
print(c1.display())  # Alice: 555-1234
print(c2.display())  # Bob: 555-5678
```

Demo: Contact Class — Watch the Live Build

Watch For:

- `class` and `def __init__` are typed first — no objects yet
- `self` as the first argument of every method
- Creating objects with `Contact("name", "phone")` syntax
- Each object holds its own data independently

```
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name
        self.phone = phone

    def display(self) -> str:
        return f"{self.name}: {self.phone}"

# Build two contacts from the same blueprint
c1 = Contact("Alice", "555-1234")
```

Lab: Contact Class

Instructions (30 minutes)

Task: Create a `Contact` class, make multiple objects, and store them in a list.

Step 1 — Define the class:

```
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        """Create a contact with a name and phone number."""
        ...  # your code here

    def display(self) -> str:
        """Return a formatted string: 'Name: Phone'."""
        ...  # your code here
```

Lab: Contact Class (continued)

Step 2 — Create objects and store in a list:

```
# Create at least 3 contacts
c1 = Contact("Alice", "555-1234")
c2 = Contact("Bob",   "555-5678")
c3 = Contact("Carol", "555-9999")

# Store them in a list
contacts = [c1, c2, c3]

# Loop and call display() on each
print("=== Contact List ===")
for contact in contacts:
    print(contact.display())
```

Expected Output:

```
=== Contact List ===
Alice: 555-1234
```

Common Pitfalls — Hour 37

Pitfall 1: Forgetting `self` in a Method Signature

```
# Bug: missing self — Python will raise TypeError
class Contact:
    def display() -> str:           # ← missing self!
        return f"{self.name}: {self.phone}"

c = Contact("Alice", "555-1234")
c.display()
# TypeError: display() takes 0 positional arguments but 1 was given
```

```
# Fix: self is ALWAYS the first parameter of every instance method
class Contact:
    def display(self) -> str:       # ← correct
        return f"{self.name}: {self.phone}"
```

> ⚠️ **Common pitfall:** Every method definition must have `self` as its first parameter — no exceptions for instance methods.

Common Pitfalls — Hour 37 (continued)

Pitfall 2: Class Attribute vs Instance Attribute Confusion

```
# Bug: defining attribute at class level instead of inside __init__
class Contact:
    name = "Unknown"    # class attribute — SHARED by all instances!
    phone = ""          # class attribute — SHARED by all instances!

c1 = Contact()
c2 = Contact()
Contact.name = "Alice"   # changes it for EVERY Contact object!
print(c1.name)           # Alice
print(c2.name)           # Alice — probably not what you wanted!
```

```
# Fix: put instance data inside __init__, assigned to self
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name    # instance attribute — unique per object
        self.phone = phone  # instance attribute — unique per object
```

Common Pitfalls — Hour 37 (continued)

Pitfall 3: Calling a Method Without an Instance

```python
# Bug: calling the method on the class instead of an object
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name
        self.phone = phone

    def display(self) -> str:
        return f"{self.name}: {self.phone}"

print(Contact.display())
# TypeError: display() missing 1 required positional argument: 'self'
```

```python
# Fix: always call a method on an instance (object)
c = Contact("Alice", "555-1234")
print(c.display())   # Alice: 555-1234
```

Optional Extensions — Hour 37

Extension 1: Add an `update_phone` Method

```
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name
        self.phone = phone

    def display(self) -> str:
        return f"{self.name}: {self.phone}"

    def update_phone(self, new_phone: str) -> None:
        """Replace phone with new_phone if non-empty."""
        if new_phone.strip():
            self.phone = new_phone.strip()

c = Contact("Alice", "555-1234")
c.update_phone("555-9999")
print(c.display())   # Alice: 555-9999

c.update_phone("")   # empty — ignored
```

Quick Check — Hour 37

Exit Ticket Question: What does `self` represent inside a method?

Model Answer: "`self` is a reference to the specific **instance** (object) that the method was called on. When you write `c1.display()`, Python automatically passes `c1` as the first argument — which is received as `self` inside the method. This is how the method knows which object's `name` and `phone` to use. Every instance method must have `self` as its first parameter so that it can access and modify the data belonging to that particular object."

Hour 38: Collections of Objects + Searching

Learning Outcomes

- Store a collection of objects in a list
- Search a list of objects by attribute value
- Update an object's attribute after retrieval
- Handle the "not found" case explicitly

A List of Objects Is Just a List

Same Rules as Any Python List

```
contacts = []

# Add objects with append
contacts.append(Contact("Alice", "555-1234"))
contacts.append(Contact("Bob",   "555-5678"))
contacts.append(Contact("Carol", "555-9999"))

# len, indexing, looping — all work exactly the same
print(len(contacts))        # 3
print(contacts[0].name)     # Alice
print(contacts[-1].phone)   # 555-9999

for c in contacts:
    print(c.display())
```

> 💡 **There is nothing special about a list of objects.** You already know how to

Searching a List of Objects

The Pattern: Loop and Compare an Attribute

```
def search_contact(contacts: list, name: str):
    """Return the Contact whose name matches, or None if not found."""
    target = name.strip().lower()
    for c in contacts:
        if c.name.strip().lower() == target:
            return c
    return None
```

Calling the Search Function

```
result = search_contact(contacts, "alice")

if result:
    print(f"Found: {result.display()}")
else:
    print("Contact not found.")
```

Updating an Object After Search

Modify the Attribute Directly on the Returned Object

```
def update_phone(contacts: list, name: str, new_phone: str) -> bool:
    """Find a contact by name and update their phone.
    Returns True if found and updated, False otherwise."""
    contact = search_contact(contacts, name)
    if contact is None:
        return False
    contact.phone = new_phone
    return True
```

Usage

```
success = update_phone(contacts, "Alice", "555-0001")
if success:
    print("Phone updated.")
else:
    print("Contact not found — no changes made.")
```

Demo: Search and Update Contact

Watch For:

- Normalizing both sides when comparing names
- Checking `if result is None` before accessing attributes
- Update goes through the returned object directly

```python
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name
        self.phone = phone

    def display(self) -> str:
        return f"{self.name}: {self.phone}"

def search_contact(contacts: list, name: str):
    target = name.strip().lower()
    for c in contacts:
        if c.name.strip().lower() == target:
            return c
    return None

# Build a small contacts list
contacts = [
    Contact("Alice", "555-1234"),
    Contact("Bob",   "555-5678"),
    Contact("Carol", "555-9999"),
```

The "Not Found" Case — Always Handle It

What Happens If You Skip the None Check?

```
# Bug: accessing attributes on None crashes the program
result = search_contact(contacts, "Zara")
print(result.display())
# AttributeError: 'NoneType' object has no attribute 'display'
```

Three Safe Patterns for Handling None

```
# Pattern 1: if result guard (most readable)
result = search_contact(contacts, "Zara")
if result:
    print(result.display())
else:
    print("Not found.")

# Pattern 2: explicit is None check (extra clarity)
result = search_contact(contacts, "Zara")
```

Lab: Refactor Contact Manager to Objects

Instructions (30 minutes)

Task: Start from your Day 9 contact manager (dict-based) or use the template below. Replace dictionaries with `Contact` objects and add search and update functions.

Starter Template:

```
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name
        self.phone = phone

    def display(self) -> str:
        return f"{self.name}: {self.phone}"

def search_contact(contacts: list, name: str):
    """Return matching Contact or None."""
    ...  # your code here
```

Lab: Refactor Contact Manager (continued)

Menu Loop to Connect Everything:

```python
def main() -> None:
    contacts = []
    print("Contact Manager (Object Version)")

    while True:
        print("\n1. Add  2. List  3. Search  4. Update  5. Quit")
        choice = input("Choice: ").strip()

        if choice == "1":
            name  = input("Name: ").strip().title()
            phone = input("Phone: ").strip()
            contacts.append(Contact(name, phone))
            print(f"Added {name}.")

        elif choice == "2":
            if not contacts:
                print("No contacts yet.")
            for c in contacts:
                print(f"  {c.display()}")

        elif choice == "3":
            name = input("Search name: ").strip()
            result = search_contact(contacts, name)
            if result:
                print(f"Found: {result.display()}")
            else:
                print("Not found.")

        elif choice == "4":
            name  = input("Name to update: ").strip()
            phone = input("New phone: ").strip()
            if update_phone(contacts, name, phone):
                print("Updated.")
            else:
                print("Contact not found.")

        elif choice == "5":
            print("Goodbye!")
```

Common Pitfalls — Hour 38

Pitfall 1: Comparing the Wrong Attribute

```python
# Bug: searching by .phone instead of .name
def search_contact(contacts: list, name: str):
    target = name.strip().lower()
    for c in contacts:
        if c.phone.strip().lower() == target:   # ← wrong attribute!
            return c
    return None
```

```python
# Fix: compare the attribute that makes semantic sense for the search
def search_contact(contacts: list, name: str):
    target = name.strip().lower()
    for c in contacts:
        if c.name.strip().lower() == target:    # ← correct
            return c
    return None
```

Optional Extensions — Hour 38

Extension: Unique Integer ID Field

```
class Contact:
    _next_id: int = 1    # class-level counter — shared across all instances

    def __init__(self, name: str, phone: str) -> None:
        self.id = Contact._next_id      # assign a unique id
        Contact._next_id += 1           # increment for next contact
        self.name = name
        self.phone = phone

    def display(self) -> str:
        return f"[{self.id}] {self.name}: {self.phone}"

# Demo
c1 = Contact("Alice", "555-1234")
c2 = Contact("Bob",   "555-5678")
c3 = Contact("Alice", "555-0000")   # duplicate name — different id!

print(c1.display())   # [1] Alice: 555-1234
print(c2.display())   # [2] Bob: 555-5678
```

Quick Check — Hour 38

Exit Ticket Question: Why might storing objects make code easier to maintain than using nested dictionaries?

Model Answer: "With nested dictionaries, every piece of code that reads or writes a contact must know the exact key names (`'name'`, `'phone'`) and trust they exist. There is nothing stopping you from creating `{'name': 'Alice'}` without a phone, or using `'phonenumber'` by accident in one place. With a `Contact` class, **the structure is enforced by the `__init__` method** — every contact is guaranteed to have both `name` and `phone`. Methods like `display()` are defined once and work the same way everywhere. Adding a new field means updating the class in one place, not hunting through every dictionary in the codebase."

Hour 39: Basic Encapsulation + Data Validation

Learning Outcomes

- Add validation logic inside class methods
- Use methods to enforce business rules in one place
- Return meaningful feedback for invalid input
- Keep UI code separate from validation logic

What Is Encapsulation?

The Core Idea

Encapsulation means bundling related **data** and **behavior** together in one place — the class.

```
# Without encapsulation — validation scattered everywhere
name  = input("Name: ").strip()
phone = input("Phone: ").strip()

# UI code validates phone — logic is in the wrong place!
digits = phone.replace("-", "").replace(" ", "")
if not digits.isdigit() or len(digits) < 7:
    print("Invalid phone.")
else:
    contacts.append({"name": name, "phone": phone})
```

```
# With encapsulation — validation lives in the class
class Contact:
```

Validation Inside a Method

A Flexible Phone Validator

```
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name
        self.phone = phone

    def display(self) -> str:
        return f"{self.name}: {self.phone}"

    def update_phone(self, new_phone: str) -> bool:
        """Update phone if valid (7–15 digits after stripping separators).
        Returns True if updated, False if invalid."""
        digits = new_phone.replace("-", "").replace(" ", "")
        digits = digits.replace("(", "").replace(")", "")
        if not digits.isdigit() or not (7 <= len(digits) <= 15):
            return False
        self.phone = new_phone
        return True
```

Demo: Validation in `update_phone`

Watch For:

- The class handles the validation rule — the `update_phone` method
- `return False` signals invalid input to the caller
- `return True` signals success — and the attribute is updated
- UI code shows the appropriate message based on the return value

```
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name
        self.phone = phone

    def display(self) -> str:
        return f"{self.name}: {self.phone}"

    def update_phone(self, new_phone: str) -> bool:
        digits = new_phone.replace("-", "").replace(" ", "")
        if not digits.isdigit() or len(digits) < 7:
            return False
        self.phone = new_phone
```

Separation of Concerns

UI Code and Validation Code Are Different Responsibilities

```python
# Contact class — owns the data and the rules
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name
        self.phone = phone

    def display(self) -> str:
        return f"{self.name}: {self.phone}"

    def update_phone(self, new_phone: str) -> bool:
        """Class owns the rule: what makes a valid phone?"""
        digits = new_phone.replace("-", "").replace(" ", "")
        return digits.isdigit() and len(digits) >= 7

    def update_name(self, new_name: str) -> bool:
        """Class owns the rule: what makes a valid name?"""
        if not new_name.strip():
            return False
        self.name = new_name.strip().title()
```

Lab: Add Validation

Instructions (30 minutes)

Task: Add at least one validation rule to your `Contact` class, then use it in your menu loop.

Step 1 — Add `update_phone` with validation:

```
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name
        self.phone = phone

    def display(self) -> str:
        return f"{self.name}: {self.phone}"

    def update_phone(self, new_phone: str) -> bool:
        """Validate then update. Return True if valid, False otherwise."""
        ...  # strip separators, check isdigit(), check length
             # if valid: update self.phone and return True
```

Lab: Add Validation (continued)

Step 2 — Add name validation (optional second rule):

```
    def update_name(self, new_name: str) -> bool:
        """Validate then update name. Returns True if updated."""
        if not new_name.strip():
            return False
        self.name = new_name.strip().title()
        return True
```

Step 3 — Connect to your menu loop:

```
elif choice == "4":   # update phone
    name = input("Contact to update: ").strip()
    result = search_contact(contacts, name)
    if result is None:
        print("Contact not found.")
    else:
        new_phone = input("New phone: ").strip()
        if result.update_phone(new_phone):
```

Common Pitfalls — Hour 39

Pitfall 1: Overly Strict Validation Rejects Valid Numbers

```
# Bug: only accepts digit-only strings — rejects common formats
def update_phone(self, new_phone: str) -> bool:
    if not new_phone.isdigit():    # "555-1234" fails — has a dash!
        return False
    self.phone = new_phone
    return True
```

```
# Fix: strip common separators before checking
def update_phone(self, new_phone: str) -> bool:
    digits = new_phone.replace("-", "").replace(" ", "")
    digits = digits.replace("(", "").replace(")", "").replace("+", "")
    if not digits.isdigit() or len(digits) < 7:
        return False
    self.phone = new_phone   # store in the original format the user typed
    return True
```

Common Pitfalls — Hour 39 (continued)

Pitfall 2: Forgetting to Check the Return Value

```
# Bug: calling update_phone but ignoring whether it succeeded
contact.update_phone(new_phone)   # validation runs... result ignored!
print(f"Updated: {contact.display()}")  # always prints — even if invalid!
```

```
# Fix: always check the boolean returned by a validation method
if contact.update_phone(new_phone):
    print(f"Updated: {contact.display()}")
else:
    print("Invalid phone number — no changes made.")
```

Pitfall 3: Putting UI Logic Inside the Class Method

```
# Bug: class method prints its own message — mixes concerns
def update_phone(self, new_phone: str) -> bool:
    digits = new_phone.replace("-", "").replace(" ", "")
    if not digits.isdigit():
```

Optional Extensions — Hour 39

Extension: Normalize Phone Storage to Digits-Only

```python
def normalize_phone(phone: str) -> str:
    """Strip all non-digit characters for consistent storage.
    Example: '(555) 123-4567' → '5551234567'
    """
    digits_only = ""
    for ch in phone:
        if ch.isdigit():
            digits_only += ch
    return digits_only


class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name
        # Normalize on the way in so storage is consistent
        self.phone = normalize_phone(phone)

    def display(self) -> str:
        # Display in a consistent formatted style
        digits = self.phone
        if len(digits) == 10:
```

Quick Check — Hour 39

Exit Ticket Question: Where is the best place to enforce validation — inside the class or in the UI code?

Model Answer: "Validation belongs **inside the class**. The class owns the data, so it should own the rules about what data is valid. If you put validation in the UI code, you must duplicate it everywhere the data is created or updated — every menu option, every input prompt. When the rule changes, you must hunt down every copy. Centralizing validation in the class means there is **one place to update**, and all callers automatically benefit. The UI code only needs to check the return value and display the appropriate message — it does not need to know *why* something is invalid."

Hour 40: Checkpoint 5 — Functions, Modules & Intro OOP

Learning Outcomes

- Demonstrate modular code organization across multiple files
- Use at least one custom class to store records
- Build a CLI application that adds, lists, and searches records

Checkpoint 5: What You Are Building

A Memory-Only CLI Organizer

You will build a **menu-driven command-line application** that:

- Stores records using a **custom class** (Contact or Task)
- Uses **at least 3 named functions**
- Imports from at least **1 custom module** (`utils.py`)
- Has **Add / List / Search** features
- Runs without crashing on typical user input
- **Does not write to or read from files** — all data lives in memory

> 🗒️ **Speaker note:** Spend 10–15 minutes reviewing the deliverables checklist before students start. Make sure everyone knows what their target structure looks like *before* they write a single line.

Checkpoint 5 Deliverables Checklist

Feature	Requirement
Menu loop	Runs until user chooses Quit
Functions	At least 3 named functions (beyond `__init__`)
Custom module	At least 1 imported helper module (e.g., `utils.py`)
Class	At least 1 class used to store a record (Task or Contact)
Add	User can add a new record
List	User can list all records
Search	User can search records by name / title
No file I/O	All data lives in memory — resets on each run

Suggested File Structure

Two-File Layout (Minimum)

```
project/
├── main.py          # Menu loop, user interaction
├── utils.py         # Shared helpers: safe_int, menu_choice, etc.
└── contact.py       # Contact class definition (or task.py, item.py)
```

Responsibilities

File	Responsibility
`main.py`	Entry point — menu loop, calls functions
`utils.py`	Reusable helpers not specific to any class
`contact.py`	`Contact` class definition only

> ⚠️ **Never name your files `random.py`, `string.py`, `os.py`, or any other**

Example: contact.py

The Class File — One Responsibility

```python
# contact.py

class Contact:
    """Stores a single contact's name and phone number."""

    def __init__(self, name: str, phone: str) -> None:
        """Create a Contact with name and phone."""
        self.name = name.strip().title()
        self.phone = phone.strip()

    def display(self) -> str:
        """Return a formatted display string."""
        return f"{self.name}: {self.phone}"

    def update_phone(self, new_phone: str) -> bool:
        """Validate and update phone. Returns True if successful."""
        digits = new_phone.replace("-", "").replace(" ", "")
        if not digits.isdigit() or len(digits) < 7:
            return False
        self.phone = new_phone.strip()
```

Example: utils.py

Reusable Helpers

```python
# utils.py

def menu_choice(options: list) -> str:
    """Display a numbered menu and return the user's chosen option.

    Loops until the user enters a valid number.
    """
    for i, option in enumerate(options, start=1):
        print(f"  {i}. {option}")
    while True:
        raw = input("Choice: ").strip()
        if raw.isdigit():
            index = int(raw) - 1
            if 0 <= index < len(options):
                return options[index]
        print(f"Please enter a number between 1 and {len(options)}.")


def confirm(prompt: str) -> bool:
    """Prompt for yes/no confirmation. Returns True for 'y' or 'yes'."""
```

Example: main.py

The Entry Point

```python
# main.py
from contact import Contact
from utils import menu_choice

def add_contact(contacts: list) -> None:
    """Prompt user and add a new Contact to the list."""
    name  = input("Name: ").strip()
    phone = input("Phone: ").strip()
    contacts.append(Contact(name, phone))
    print(f"Added {name.strip().title()}.")

def list_contacts(contacts: list) -> None:
    """Print all contacts, or a message if none exist."""
    if not contacts:
        print("No contacts yet.")
        return
    print("=== Contacts ===")
    for c in contacts:
        print(f"  {c.display()}")

def search_contacts(contacts: list) -> None:
    """Prompt for a name and display the matching contact."""
    name = input("Search name: ").strip().lower()
    for c in contacts:
        if c.name.lower() == name:
            print(f"Found: {c.display()}")
            return
    print(f"No contact named '{name.title()}' found.")

def main() -> None:
    contacts = []
    print("=== CLI Organizer — Checkpoint 5 ===")
    options = ["Add Contact", "List Contacts", "Search", "Quit"]

    while True:
        print()
        choice = menu_choice(options)
        if choice == "Add Contact":
            add_contact(contacts)
        elif choice == "List Contacts":
            list_contacts(contacts)
        elif choice == "Search":
            search_contacts(contacts)
        elif choice == "Quit":
```

Run It — Expected Interaction

```
=== CLI Organizer — Checkpoint 5 ===

  1. Add Contact
  2. List Contacts
  3. Search
  4. Quit
Choice: 1
Name: alice smith
Phone: 555-1234
Added Alice Smith.

  1. Add Contact
  2. List Contacts
  3. Search
  4. Quit
Choice: 1
Name: bob jones
Phone: 555-5678
Added Bob Jones.

  1. Add Contact
  2. List Contacts
  3. Search
  4. Quit
Choice: 2
=== Contacts ===
  Alice Smith: 555-1234
  Bob Jones: 555-5678

  1. Add Contact
  2. List Contacts
  3. Search
  4. Quit
Choice: 3
Search name: alice
Found: Alice Smith: 555-1234

  1. Add Contact
  2. List Contacts
  3. Search
  4. Quit
Choice: 4
Goodbye!
```

Lab: Checkpoint 5 — CLI Organizer in Memory

Instructions (50 minutes)

Build a menu-driven CLI Organizer that meets all checklist items.

Minimum Requirements:

- `main.py` — entry point with menu loop
- `utils.py` — at least 2 reusable helper functions
- `contact.py` (or `task.py`) — your custom class
- Menu options: Add / List / Search / Quit
- At least 3 named functions beyond `__init__`
- No crashes on empty input, missing contacts, or invalid phone numbers

Time Targets:

Common Pitfalls — Hour 40

Pitfall 1: ImportError Because Files Are in Different Folders

```
project/
├── main.py
└── helpers/
    └── utils.py    # ← wrong!  utils.py is not in the same folder
```

```
# Bug in main.py:
from utils import menu_choice
# ModuleNotFoundError: No module named 'utils'
```

```
project/
├── main.py
├── utils.py        # ← correct: same folder as main.py
└── contact.py
```

> ⚠️ **Common pitfall:** All files must be in the **same directory**. Run `python`

Common Pitfalls — Hour 40 (continued)

Pitfall 2: Naming a File After a Standard Library Module

```
# Bug: you created 'string.py' or 'os.py' in your project folder
import os        # Python finds YOUR os.py instead!
os.getcwd()      # AttributeError: module 'os' has no attribute 'getcwd'
```

```
# Fix: use project-specific names
# ✅ contact.py, task.py, utils.py, helpers.py, organizer.py
# ❌ string.py, os.py, math.py, json.py, random.py, pathlib.py
```

Pitfall 3: Forgetting `self` in Class Method Definitions

```
# Bug: self missing — TypeError on any method call
class Contact:
    def display() -> str:      # ← no self!
        return f"{self.name}: {self.phone}"
```

Common Pitfalls — Hour 40 (continued)

Pitfall 4: Running contact.py Instead of main.py

```
# Bug: running the class file directly
$ python contact.py
# No output — there is no code to run at the module level!

# Fix: always run your entry point
$ python main.py
```

Pitfall 5: Circular Imports

```
# Bug: contact.py imports from main.py, and main.py imports from contact.py
# Each file tries to import the other — Python can't finish either!
```

```
# Fix: keep the import flow one-directional
# main.py imports from contact.py and utils.py
# contact.py and utils.py import from neither main.py nor each other
```

Optional Extensions — Hour 40

Extension 1: Delete a Contact

```
def delete_contact(contacts: list, name: str) -> bool:
    """Remove the first contact whose name matches.
    Returns True if deleted, False if not found."""
    target = name.strip().lower()
    for i, c in enumerate(contacts):
        if c.name.lower() == target:
            removed = contacts.pop(i)
            print(f"Deleted: {removed.display()}")
            return True
    return False
```

Extension 2: Update Phone From the Menu

```
def update_contact_phone(contacts: list) -> None:
    """Prompt for a name, find the contact, and update the phone."""
    name = input("Contact name to update: ").strip()
```

Quick Check — Hour 40

Exit Ticket Question: Point to one place in your code where using a function reduced duplication.

Model Answer (example): "I used `search_contacts()` in both my Update and Delete features. Without this function, I would have copied the same loop and `.lower()` comparison into two different `elif` blocks. By extracting it once into a named function, both features call the same logic — if I ever change how searching works (e.g., add partial matching), I only need to update it in one place. This is the **Don't Repeat Yourself (DRY)** principle in action."

Session 10 Recap

What We Covered Today

Hour	Topic	Key Takeaway
37	Intro to classes — objects, attributes, methods	A class is a blueprint; objects are independent instances built from it
38	Collections of objects + searching	Loop and compare attributes; normalize strings; always check for `None`
39	Basic encapsulation + data validation	Validation logic belongs inside the class; UI only checks the return value
40	Checkpoint 5 — Functions, modules & OOP	One file = one responsibility; import flow is one-directional

Session 10 Key Concepts

The Four Big Ideas

1. Classes enforce structure A `Contact` class guarantees every contact has `name` and `phone`. You can't accidentally create an incomplete record.

2. Objects are independent Creating `c1 = Contact("Alice", ...)` and `c2 = Contact("Bob", ...)` gives you two separate objects. Changing `c1.name` never touches `c2`.

3. Encapsulation keeps rules in one place Validation in the class means every caller benefits automatically. Change the rule once — it takes effect everywhere.

4. Modules separate responsibilities `contact.py` knows about contacts. `utils.py` knows about helpers. `main.py` knows about the user interface. None of them needs to know what the others do internally.

Session 10 Key Terms Reference

Term	Definition
class	A blueprint that defines the structure and behavior of objects
object / instance	A concrete value built from a class blueprint
attribute	A piece of data stored on an instance (`self.name`)
method	A function defined inside a class; always receives `self`
`self`	A reference to the specific instance a method was called on
`__init__`	The constructor — called automatically when an object is created
encapsulation	Bundling data and the methods that operate on it in one class
validation	Checking that input meets requirements before storing it
module	A `.py` file containing functions, classes, or variables for reuse

No-Go Topics for Basics Scope

> 🚫 **Scope guardrail (Basics):** The following are **Advanced course** topics. Do not introduce them during this session.

Topic	Why It Is Advanced
Inheritance and subclassing	Requires understanding of the method resolution order (MRO)
Dunder methods beyond `__init__`	`__str__`, `__repr__`, `__eq__`, `__len__` — Advanced OOP
`@property` decorators	Computed attributes — syntactic sugar requiring decorator knowledge
`@classmethod` / `@staticmethod`	Alternative constructors and utilities — Advanced design patterns

Session 10 Common Patterns Reference

Define a Class and Create Objects

```
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name = name
        self.phone = phone

    def display(self) -> str:
        return f"{self.name}: {self.phone}"

contacts = [
    Contact("Alice", "555-1234"),
    Contact("Bob",   "555-5678"),
]
for c in contacts:
    print(c.display())
```

Search a List of Objects

Looking Ahead — Day 11

Next Session Builds On Today

Hour 41: File I/O — Reading Files

- `open()`, the `with` statement, reading lines one at a time
- Process file data using the functions and classes you built today

Hour 42: File I/O — Writing and Appending

- Writing new files, appending to existing files
- Save your Contact list to disk; reload it on start

Hour 43: Working With CSV Files

- `csv.reader`, `csv.writer` — structured text data
- Load contacts from a `.csv` file into `Contact` objects